8

Building a Weather App for Multiple Form Factors

.NET MAUI isn't just for creating apps for phones; it can also be used to create apps for tablets and desktop computers. In this chapter, we will build an app that will work on all of these platforms and optimize the user interface for each form factor. As well as using three different form factors, we are also going to be working on four different operating systems: iOS, macOS, Android, and Windows.

The following topics will be covered in this chapter:

- Using `FlexLayout` in .NET MAUI
- Using `VisualStateManager`
- Using different views for different form factors
- Using behaviors

Let's get started!

Technical requirements

To work on this project, we need to have Visual Studio for Mac or PC installed, as well as the necessary .NET MAUI components. See *Chapter 1*, *Introduction to .NET MAUI*, for more details on how to set up your environment. To build an iOS app using Visual Studio for PC, you need to have a Mac connected. If you don't have access to a Mac at all, you can choose to just work on the Windows and Android parts of this project. Similarly, if you only have a Mac, you can choose to work on only the iOS and Android parts of this project.

You can find the full source for the code in this chapter at **https://github.com/PacktPubliching/MAUI-Projects-3rd-Edition**.

Project overview

Applications for iOS and Android can run on both phones and tablets. Often, apps are just optimized for phones. In this chapter, we will build an app that will work on different form factors, but we aren't going to stick to just phones and tablets – we are going to target desktop computers as well. The desktop version will be for **Window UI Library** (**WinUI**) and macOS via Mac Catalyst.

The app that we are going to build is a weather app that displays the weather forecast based on the location of the user. For this chapter, we will be referencing Visual Studio for Mac in the instructions. If you are using Visual Studio for Windows, you should be able to follow along. Use one of the other chapters for reference if you need help.

Building the weather app

It's time to start building the app. Create a new blank .NET MAUI app using the following steps for Visual Studio for Mac:

1. Open Visual Studio for Mac and click on **New**:

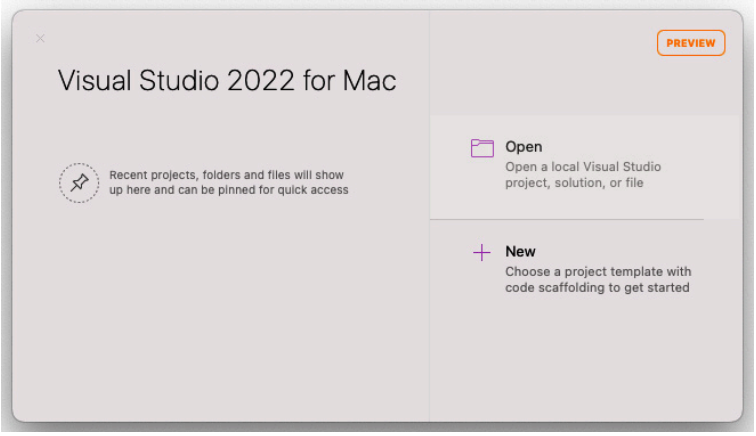

Figure 8.1 – Visual Studio 2022 for Mac start screen

2. In the **Choose a template for your new project** dialog, use the **.NET MAUI App** template, which is under **Multiplatform** | **App**, then click **Continue**:

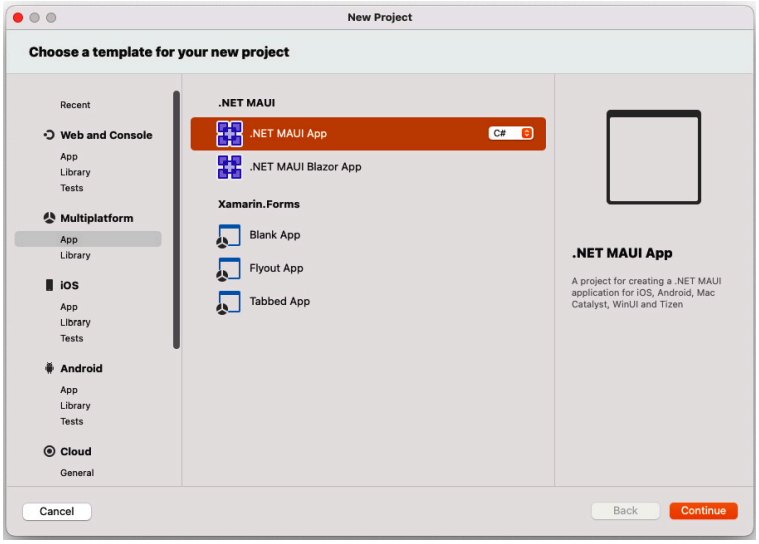

Figure 8.2 – New project

3. In the **Configure your new .NET MAUI App** dialog, ensure the **.NET 7.0** target framework is selected, then click **Continue**:

Figure 8.3 – Choosing the target framework

4. In the **Configure your new .NET MAUI App** dialog, name the project `Weather`, then click **Create**:

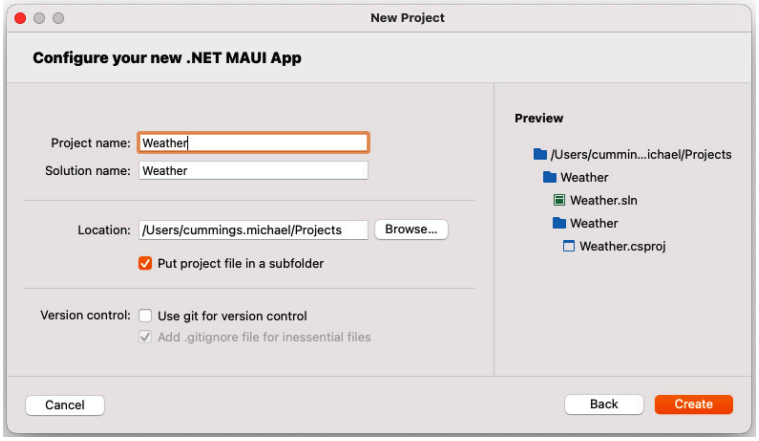

Figure 8.4 – Naming the new app

If you run the app now, you should see something like the following:

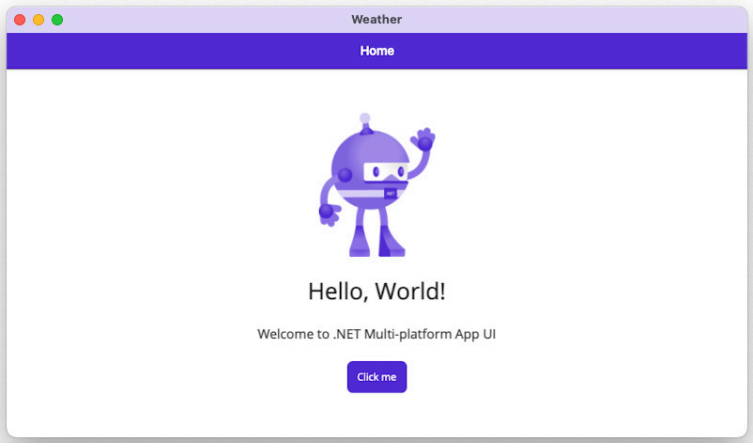

Figure 8.5 – Weather app on macOS

Now that we have created the project from a template, it's time to start coding!

Creating models for the weather data

Before we write the code to fetch data from the external weather service, we will create models to deserialize the results from the service. We will do this so that we have a common model that we can use to return data from the service.

As the data source for this app, we will use an external Weather API. This project will use **OpenWeatherMap**, a service that offers a couple of free APIs. You can find this service at **https://openweathermap.org/api**. We will use the **5 day / 3 hour forecast** service in this project, which provides a 5-day forecast in 3-hour intervals. To use the OpenWeatherMap API, we have to create an account to get an API key. If you don't want to create an API key, you can mock the data instead.

Follow the instructions at **https://home.openweathermap.org/users/sign_up** to create your account and get your API key, which you will need to call the API.

The easiest way to generate models to use when we are deserializing results from the service is to make a call to the service either in the browser or with a tool (such as **Postman**) to see the structure of the JSON. For OpenWeather's 5-day/3-hour forecast, you can use `https://api.openweathermap.org/data/2.5/forecast?lat=44.34&lon=10.99&appid={API key}` in your browser, replacing `{API KEY}` with your API key.

NOTE

If you got a 401 error, please wait a couple of hours before you can use your API, as mentioned at **https://openweathermap.org/faq#error401**.

We can create classes manually or use a tool that can generate C# classes from the JSON. One tool that can be used is **quicktype**, which can be found at **https://quicktype.io/**. Just paste the output from the API call into quicktype to generate your C# models.

If you generate them using a tool, make sure you set the namespace to `Weather.Models`.

As mentioned previously, you can also create these models manually. We will describe how to do this in the next section.

Adding the Weather API models manually

If you wish to add the models manually, then go through the following instructions. We will be adding a single code file called `WeatherData.cs`, which will contain multiple classes:

1. Create a folder called `Models`.
2. Add a file called `WeatherData.cs` to the newly created folder.
3. Add the following code to the `WeatherData.cs` file:

```
using System.Collections.Generic;
namespace Weather.Models
{
```

```
public class Main
{
    public double temp { get; set; }
    public double temp_min { get; set; }
    public double temp_max { get; set; }
    public double pressure { get; set; }
    public double sea_level { get; set; }
    public double grnd_level { get; set; }
    public int humidity { get; set; }
    public double temp_kf { get; set; }
}
public class Weather
{
    public int id { get; set; }
    public string main { get; set; }
    public string description { get; set; }
    public string icon { get; set; }
}
public class Clouds
{
    public int all { get; set; }
}
public class Wind
{
    public double speed { get; set; }
    public double deg { get; set; }
}
public class Rain
{
}
public class Sys
{
    public string pod { get; set; }
}
public class List
{
    public long dt { get; set; }
    public Main main { get; set; }
    public List<Weather> weather { get; set; }
    public Clouds clouds { get; set; }
    public Wind wind { get; set; }
    public Rain rain { get; set; }
    public Sys sys { get; set; }
    public string dt_txt { get; set; }
}
public class Coord
{
    public double lat { get; set; }
    public double lon { get; set; }
}
public class City
{
    public int id { get; set; }
    public string name { get; set; }
    public Coord coord { get; set; }
    public string country { get; set; }
}
public class WeatherData
{
    public string cod { get; set; }
    public double message { get; set; }
```

```
        public int cnt { get; set; }
        public List<List> list { get; set; }
        public City city { get; set; }
    }
}
```

As you can see, there are quite a lot of classes. These classes map directly to the response we get from the service. In most cases, you only want to use these classes when communicating with the service. To represent the data in your app, you will need to use a second set of classes that exposes only the information you need in your app.

Adding the app-specific models

In this section, we will create the models that our app will translate the Weather API models into. Let's start by adding the **`WeatherData`** class (unless you created this manually in the preceding section):

1. Create a new folder called **`Models`** in the **`Weather`** project.
2. Add a new file called **`WeatherData.cs`**.
3. Paste the generated code from quicktype or write the code for the classes based on the JSON. If code other than the properties is generated, ignore it and just use the properties.
4. Rename **`MainClass`** (this is what **quicktype** names the root object) **`WeatherData`**.

Now, we will create models based on the data we are interested in. This will make the rest of the code more loosely coupled to the data source.

Adding the ForecastItem model

The first model we are going to add is **`ForecastItem`**, which represents a specific forecast for a point in time. We can do this as follows:

1. In the **`Weather`** project and the **`Models`** folder, create a new class called **`ForecastItem`**.
2. Add the following code:

```
using System;
using System.Collections.Generic;
namespace Weather.Models
{
    public class ForecastItem
    {
        public DateTime DateTime { get; set; }
        public string TimeAsString => DateTime.ToShortTimeString();
        public double Temperature { get; set; }
        public double WindSpeed { get; set; }
        public string Description { get; set; }
        public string Icon { get; set; }
    }
}
```

Now that we have a model for each forecast, we need a container model that will group **`ForecastItems`** by **`City`**.

Adding the Forecast model

In this section, we'll create a model called **`Forecast`** that will keep track of a single forecast for a city. The **`Forecast`** model keeps a list of multiple **`ForeCastItem`** objects, each representing a forecast for a specific point in time. Let's set this up:

1. Create a new class called **`Forecast`** in the **`Models`** folder.
2. Add the following code:

```
using System;
using System.Collections.Generic;
namespace Weather.Models;
public class Forecast
{
    public string City { get; set; }
    public List<ForecastItem> Items { get; set; }
}
```

Now that we have our models for both the Weather API and the app, we need to fetch data from the Weather API.

Creating a service to fetch the weather data

To make it easier to change the external weather service and to make the code more testable, we will create an interface for the service. Here's how we can go about it:

1. Create a new folder called **`Services`**.
2. Create a new **`public interface`** called **`IWeatherService`**.
3. Add a method for fetching data based on the location of the user, as shown in the following code. Name the method **`GetForecastAsync`**:

```
using System.Threading.Tasks;
using Weather.Models;
namespace Weather.Services;
public interface IWeatherService
{
    Task<Forecast> GetForecastAsync(double latitude, double longitude);
}
```

Now that we have an interface, we can create an implementation for it, as follows:

1. In the **`Services`** folder, create a new class called **`OpenWeatherMapWeatherService`**.
2. Implement the interface and add the **`async`** keyword to the **`GetForecastAsync`** method:

```
using System;
using System.Globalization;
using Weather.Models;
using System.Text.Json;
namespace Weather.Services;
```

```
public class OpenWeatherMapWeatherService : IWeatherService
{
    public async Task<Forecast> GetForecastAsync(double latitude, double longitude)
    {
    }
}
```

Before we call the OpenWeatherMap API, we need to build a URI for the call to the Weather API. This will be a GET call, and the latitude and longitude of the position will be added as query parameters. We will also add the API key and the language that we would like the response to be in. Let's set this up:

1. Open the **`OpenWeatherMapWeatherService`** class.
2. Add the highlighted code in the following code snippet to the **`OpenWeatherMap WeatherService`** class:

```
public async Task<Forecast> GetForecastAsync(double latitude, double longitude)
{
    var language = CultureInfo.CurrentUICulture.TwoLetterISOLanguageName;
    var apiKey = "{AddYourApiKeyHere}";
    var uri = $"https://api.openweathermap.org/data/2.5/forecast?lat={latitude}&lon={longitude}
}
```

Replace the **`{AddYourApiKeyHere}`** with the key you obtained from the *Creating models for the weather data* section

To deserialize the JSON that we will get from the external service, we will use **`System.Text.JSON`**.

To make a call to the **`Weather`** service, we will use the **`HttpClient`** class and the **`GetStringAsync`** method, as follows:

1. Create a new instance of the **`HttpClient`** class.
2. Call **`GetStringAsync`** and pass the URL as the argument.
3. Use the **`JsonSerializer`** class and the **`DeserializeObject`** method from **`System.Text.Json`** to convert the JSON string into an object.
4. Map the **`WeatherData`** object to a **`Forecast`** object.
5. The code for this should look like the highlighted code shown in the following snippet:

```
public async Task<Forecast> GetForecastAsync(double latitude, double longitude)
{
    var language = CultureInfo.CurrentUICulture.
TwoLetterISOLanguageName;
    var apiKey = "{AddYourApiKeyHere}";
    var uri = $"https://api.openweathermap.org/data/2.5/forecast?lat={latitude}&lon={longitude}
    var httpClient = new HttpClient();
    var result = await httpClient.GetStringAsync(uri);
    var data = JsonSerializer.Deserialize<WeatherData>(result);
    var forecast = new Forecast()
    {
        City = data.city.name,
        Items = data.list.Select(x => new ForecastItem()
        {
```

```
            DateTime = ToDateTime(x.dt),
            Temperature = x.main.temp,
            WindSpeed = x.wind.speed,
            Description = x.weather.First().description,
            Icon = $"http://openweathermap.org/img/w/{x.weather.First().icon}.png"
        }).ToList()
    };
    return forecast;
}
```

PERFORMANCE TIP

To optimize the performance of the app, we can use **`HttpClient`** *as a singleton and reuse it for all network calls in the application. The following information is from Microsoft's documentation: "HttpClient is intended to be instantiated once and reused throughout the life of an application. Instantiating an HttpClient class for every request will exhaust the number of sockets available under heavy loads. This will result in SocketException errors." This can be found at* **https://learn.microsoft.com/en-gb/dotnet/api/system.net.http.httpclient?view=netstandard-2.0**.

In the preceding code, we have a call to a **`ToDateTime`** method, which is a method that we will need to create. This method converts the date from a Unix timestamp into a **`DateTime`** object, as shown in the following code:

```
private DateTime ToDateTime(double unixTimeStamp)
{
    DateTime dateTime = new DateTime(1970, 1, 1, 0, 0, 0, 0, DateTimeKind.Utc);
    dateTime = dateTime.AddSeconds(unixTimeStamp).ToLocalTime();
    return dateTime;
}
```

PERFORMANCE TIP

By default, **`HttpClient`** *uses the Mono implementation of* **`HttpClient`** *(iOS and Android). To increase performance, we can use a platform-specific implementation instead. For iOS, use* **`NSUrlSession`**. *This can be set in the project settings of the iOS project under the* **iOS Build** *tab. For Android, use* **Android**. *This can be set in the project settings of the Android project under* **Android Options** | **Advanced**.

Configuring the application platforms so that they use location services

To be able to use location services, we need to carry out some configuration on each platform.

Configuring the iOS platform so that it uses location services

To use location services in an iOS app, we need to add a description to indicate why we want to use the location in the **`info.plist`** file. In this app, we only need to get the location when we are using the app, so we only need to add a description for this. Let's set this up:

1. Open **`info.plist`** in **`Platforms/iOS`** with **XML (Text) Editor**.
2. Add the **`NSLocationWhenInUseUsageDescription`** key using the following code:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>We are using your location to find a forecast for you</string>
```

Configuring the Android platform so that it uses location services

For Android, we need to set up the app so that it requires the following two permissions:

- **ACCESS_COARSE_LOCATION**
- **ACCESS_FINE_LOCATION**

We can set this in the **`AndroidManifest.xml`** file, which can be found in the **`Platforms\Android\`** folder. However, we can also set this in the project properties on the **Android Manifest** tab, as shown in the following screenshot:

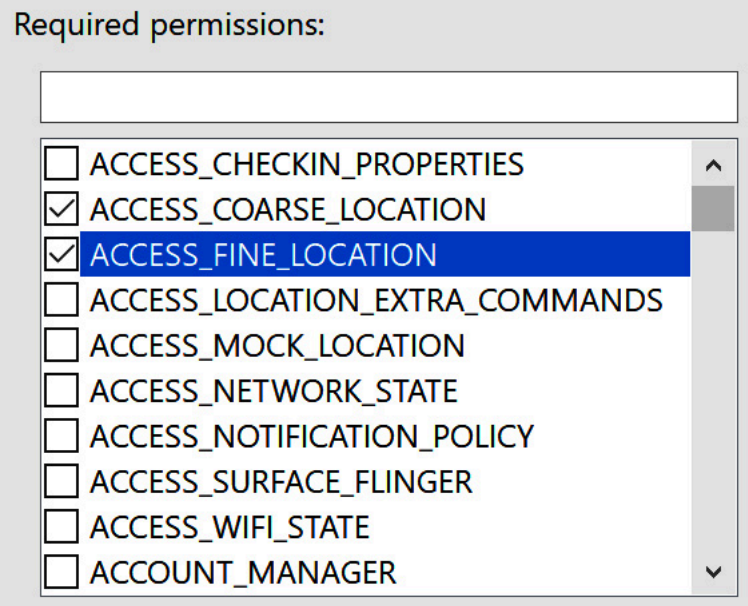

Figure 8.6 – Selecting location permissions

Configuring the WinUI platform so that it uses location services

Since we will be using location services in the WinUI platform, we need to add the **`Location`** capability under **Capabilities** in the **`Package.appxmanifest`** file of the project, which is located in the **`Platforms/Windows`** folder, as shown in the following screenshot:

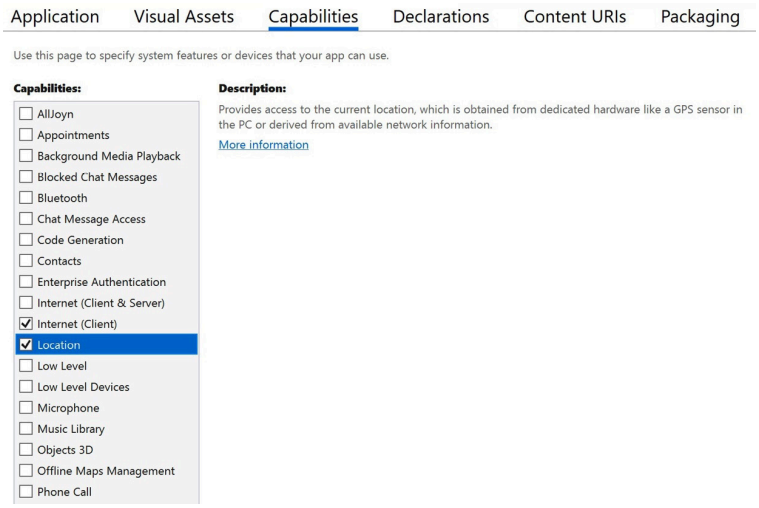

Figure 8.7 – Adding a location to the WinUI app

Creating the ViewModel class

Now that we have a service that's responsible for fetching weather data from the external weather source, it's time to create a **`ViewModel`**. First, however, we will create a base view model where we can put the code that can be shared between all the **`ViewModels`** of the app. Let's set this up:

1. Create a new folder called **`ViewModels`**.
2. Create a new class called **`ViewModel`**.
3. Make the new class **`public`** and **`abstract`**.
4. Add a package reference to CommunityToolkit.MVVM:

```
public abstract partial class ViewModel : ObservableObject
{
}
```

We now have a base view model. We can use this for the view model that we're about to create.

Now, it's time to create **`MainViewModel`**, which will be the ViewModel for our **`MainView`** in the app. Perform the following steps to do so:

1. In the **`ViewModels`** folder, create a new class called **`MainViewModel`**.
2. Add the abstract **`ViewModel`** class as a base class.
3. Because we are going to use constructor injection, we will add a constructor with the **`IWeatherService`** interface as a parameter.
   Create a **`read-only private`** field. We will use this to store the **`IweatherService`** instance:

```
public class MainViewModel : ViewModel
{
    private readonly IWeatherService weatherService;
    public MainViewModel(IWeatherService weatherService)
    {
        this.weatherService = weatherService;
```

```
    }
}
```

`MainViewModel` takes any object that implements `IWeatherService` and stores a reference to that service in a field. We will be adding functionality that will fetch weather data in the next section.

Getting the weather data

Now, we will create a new method for loading the data. This will be a three-step process. First, we will get the location of the user. Once we have this, we can fetch data related to that location. The final step is to prepare the data that the views can consume to create a user interface for the user.

To get the location of the user, we will use the `Geolocation` class, which exposes methods that can fetch the location of the user. Perform the following steps:

1. Create a new method called `LoadDataAsync`. Make it an asynchronous method that returns `Task`.
2. Use the `GetLocationAsync` method on the `Geolocation` class to get the location of the user.
3. Pass the latitude and longitude from the result of the `GetLocationAsync` call and pass it to the `GetForecast` method on the object that implements `IWeatherService` using the following code:

```
public async Task LoadDataAsync()
{
    var location = await Geolocation.GetLocationAsync();
    var forecast = await weatherService.GetForecastAsync(location.Latitude, location.Longitude)
}
```

Now that we can get data from the service, we need to structure it for our user interface by grouping the individual data items.

Grouping the weather data

When we present the weather data, we will group it by day so that all of the forecasts for one day will be under the same header. To do this, we will create a new model called `ForecastGroup`. To make it possible to use this model with the .NET MAUI `CollectionView`, it has to have an `IEnumerable` type as the base class. Let's set this up:

1. Create a new class called `ForecastGroup` in the `Models` folder.
2. Add `List<ForecastItem>` as the base class for the new model.
3. Add an empty constructor and a constructor that has a list of `ForecastItem` instances as a parameter.
4. Add a `Date` property.
5. Add a property, `DateAsString`, that returns the `Date` property as a short date string.
6. Add a property, `Items`, that returns the list of `ForecastItem` instances, as shown in the following code:

```
using System;
namespace Weather.Models;
public class ForecastGroup : List<ForecastItem>
{
    public ForecastGroup() { }
    public ForecastGroup(IEnumerable<ForecastItem> items)
    {
        AddRange(items);
    }
    public DateTime Date { get; set; }
    public string DateAsString => Date.ToShortDateString();
    public List<ForecastItem> Items => this;
}
```

When we have done this, we can update **`MainViewModel`** with two new properties, as follows:

1. Create a private field called **`city`** with the **`ObservableProperty`** attribute for the name of the city we are fetching the weather data.
2. Create a private field called **`days`** with the **`ObservableProperty`** attribute that will contain the grouped weather data.
3. The **`MainViewModel`** class should look like the highlighted code shown in the following snippet:

```
public partial class MainViewModel : ViewModel
{
    [ObservableProperty]
    private string city;
    [ObservableProperty]
    private ObservableCollection<ForecastGroup> days;
    // Rest of the class is omitted for brevity
}
```

Now, we are ready to group the data. We will do this in the **`LoadDataAsync`** method. We will loop through the data from the service and add items to various groups, as follows:

1. Create an **`itemGroups`** variable of the **`List<ForecastGroup>`** type.
2. Create a **`foreach`** loop that loops through all the items in the **`forecast`** variable.
3. Add an **`if`** statement that checks whether the **`itemGroups`** property is empty. If it is empty, add a new **`ForecastGroup`** to the variable and continue to the next item in the item list.
4. Use the **`SingleOrDefault`** method (this is an extension method from **`System.Linq`**) on the **`itemGroups`** variable to get a group based on the date of the current **`ForecastItem`**. Add the result to a new variable, **`group`**.
5. If the **`group`** property is **`null`**, then there is no group with the current day in the list of groups. If this is the case, a new **`ForecastGroup`** should be added to the list in the **`itemGroups`** variable. The code will continue executing until it gets to the next **`forecast`** item in the **`forecast.Items`** list. If a group is found, it should be added to the list in the **`itemGroups`** variable.

6. After the **foreach** loop, set the **Days** property with a new **ObservableCollection <ForecastGroup>** and use the **itemGroups** variable as an argument in the constructor.
7. Set the **City** property to the **City** property of the **forecast** variable.
8. The **LoadDataAsync** method should now look as follows:

```
public async Task LoadDataAsync()
{
    var location = await Geolocation.GetLocationAsync();
    var forecast = await weatherService.GetForecastAsync(location.Latitude, location.Longitude)
    var itemGroups = new List<ForecastGroup>();
    foreach (var item in forecast.Items)
    {
        if (!itemGroups.Any())
        {
            itemGroups.Add(new ForecastGroup(new List<ForecastItem>() { item })
            {
                Date = item.DateTime.Date
            });
            continue;
        }
        var group = itemGroups.SingleOrDefault(x => x.Date == item.DateTime.Date);
        if (group == null)
        {
            itemGroups.Add(new ForecastGroup(new List<ForecastItem>() { item })
            {
                Date = item.DateTime.Date
            });
            continue;
        }
        group.Items.Add(item);
    }
    Days = new ObservableCollection<ForecastGroup>(itemGroups);
    City = forecast.City;
}
```

TIP

Don't use the **Add** *method on* **ObservableCollection** *when you want to add more than a couple of items. It is better to create a new instance of* **ObservableCollection** *and pass a collection to the constructor. The reason for this is that every time you use the* **Add** *method, you will have a binding to it from the view, which will cause the view to be rendered. We will get better performance if we avoid using the* **Add** *method.*

Creating the view for tablets and desktop computers

The next step is to create the view that we will use when the app is running on a tablet or a desktop computer. Let's set this up:

1. Create a new folder in the **Weather** project called **Views**.
2. In the **Views** folder, create a new folder called **Desktop**.
3. Create a new **.NET MAUI ContentPage (XAML)** file called **MainView** in the **Views\Desktop** folder:

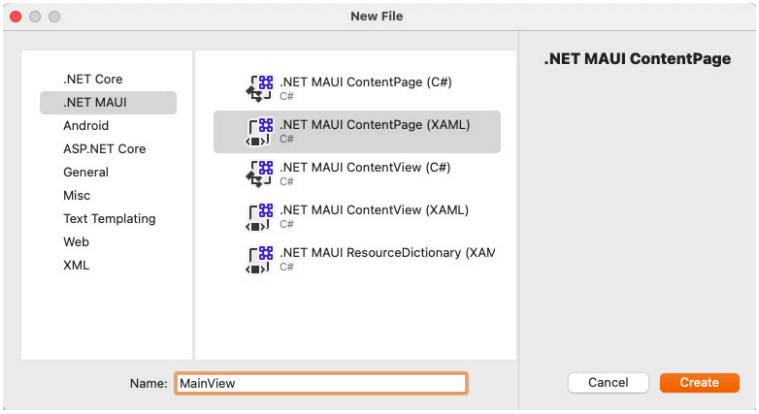

Figure 8.8 – Adding a .NET MAUI XAML ContentPage

4. Pass an instance of **`MainViewModel`** in the constructor of the view to set **`BindingContext`**, as shown in the following code:

```
public MainView (MainViewModel mainViewModel)
{
    InitializeComponent ();
    BindingContext = mainViewModel;
}
```

   Later, in the *Adding services and ViewModels to dependency injection* section, we will configure dependency injection to provide the instances for us.

To trigger the **`LoadDataAsync`** method in **`MainViewModel`**, call the **`LoadDataAsync`** method by overriding the **`OnNavigatedTo`** method on the main thread. We need to make sure that the call is executed on the UI thread since it will interact with the user interface.

To do this, perform the following steps:

1. Open the **`MainView.xaml.cs`** file in the **`Views\Desktop`** folder.
2. Create an override of the **`OnNavigatedTo`** method.
3. Add the highlighted code shown in the following snippet to the **`OnNavigateTo`** method:

```
protected override void OnNavigatedTo(NavigatedToEventArgs args)
{
    base.OnNavigatedTo(args);
    if (BindingContext is MainViewModel viewModel)
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {
            await viewModel.LoadDataAsync();
        });
    }
}
```

In the **`MainView`** XAML file, add a binding for the **`Title`** property of **`ContentPage`** to the **`City`** property in **`ViewModel`**, as follows:

1. Open the **`MainView.xaml`** file in the **`Views\Desktop`** folder.

2. Add the **`Title`** binding to the **`ContentPage`** element, as highlighted in the following code snippet:

```
<ContentPage
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Weather.Views.Desktop.MainView"
    Title="{Binding City}">
```

In this next section, we will use **`FlexLayout`** to render the data from the ViewModel onto the screen.

Using FlexLayout

In .NET MAUI, we can use **`CollectionView`** or **`ListView`** if we want to show a collection of data. Using both **`CollectionView`** and **`ListView`** works great in most cases, and we will use **`CollectionView`** later in this chapter, but **`ListView`** can only show data vertically. In this app, we want to show data in both directions. In the vertical direction, we will have the days (we group forecasts based on days), while in the horizontal direction, we will have the forecasts within a particular day. We also want the forecasts within a day to wrap if there is not enough space for all of them in one row. **`CollectionView`** can show data in a horizontal direction, but it will not wrap. With **`FlexLayout`**, we can add items in both directions and we can use **`BindableLayout`** to bind items to it. When we use **`BindableLayout`**, we will use **`ItemSource`** and **`ItemsTemplate`** as attached properties.

Perform the following steps to build the view:

1. Add **`Grid`** as the root view of the page.
2. Add **`ScrollView`** to **`Grid`**. We need this to be able to scroll if the content is higher than the height of the page.
3. Add **`FlexLayout`** to **`ScrollView`** and set the direction to **`Column`** so that the content will be in a vertical direction.
4. Add a binding to the **`Days`** property in **`MainViewModel`** using **`BindableLayout.ItemsSource`**.
5. Set **`DataTemplate`** to the content of **`ItemsTemplate`**, as shown in the following code:

```
<Grid>
  <ScrollView BackgroundColor="Transparent">
    <FlexLayout BindableLayout.ItemsSource="{Binding Days}" Direction="Column">
      <BindableLayout.ItemTemplate>
        <DataTemplate>
          <!--Content will be added here -->
        </DataTemplate>
      </BindableLayout.ItemTemplate>
    </FlexLayout>
  </ScrollView>
</Grid>
```

The content for each item will be a header with the date and a horizontal **FlexLayout** with the forecasts for the day. Let's set this up:

1. Open the **MainView.xaml** file.
2. Add **StackLayout** so that the children we add to it will be placed in a vertical direction.
3. Add **ContentView** to **StackLayout** with **Padding** set to **10** and **BackgroundColor** set to **#9F5010**. This will be the header. The reason we need **ContentView** is that we want to have padding around the text.
4. Add **Label** to **ContentView** with **TextColor** set to **White** and **FontAttributes** set to **Bold**.
5. Add a binding to **DateAsString** for the **Text** property of **Label**.
6. The code should be placed at the **<!-- Content will be added here -->** comment and should look as follows:

```
<StackLayout>
  <ContentView Padding="10" BackgroundColor="#9F5010">
    <Label Text="{Binding DateAsString}" TextColor="White" FontAttributes="Bold" />
  </ContentView>
</StackLayout>
```

Now that we have the date in the user interface, we need to add a **FlexLayout** property, which will repeat through any items in **MainViewModel**. Perform the following steps to do so:

1. Add **FlexLayout** after the **</ContentView>** tag but before the **</StackLayout>** tag.
2. Set **JustifyContent** to **Start** to set the items so that they're added from the left-hand side, without distributing them over the available space.
3. Set **AlignItems** to **Start** to set the content to the left of each item in **FlexLayout**, as shown in the following code:

```
<FlexLayout BindableLayout.ItemsSource="{Binding Items}" Wrap="Wrap" JustifyContent="Start" Ali
</FlexLayout>
```

After defining **FlexLayout**, we need to provide an **ItemsTemplate** property, which defines how each item in the list should be rendered. Continue adding the XAML directly under the **<FlexLayout>** tag you just added, as follows:

1. Set the **ItemsTemplate** property to **DataTemplate**.
2. **FillDataTemplate** with elements, as shown in the following code:

TIP

If we want to add formatting to a binding, we can use **StringFormat**. *In this case, we want to add the degree symbol after the temperature. We can do this by using the* **{Binding Temperature, StringFormat='{0}° C'}** *phrase. With the* **StringFormat** *property of the binding, we can format data with the same arguments that we would use if we were to do this in C#. This is the same as* **string.Format("{0}° C", Temperature)** *in C#. We can*

also use it to format a date; for example, **`{Binding Date, StringFormat='yyyy'}`**. *In C#, this would look like* **`Date.ToString("yyyy")`**.

```
<BindableLayout.ItemTemplate>
  <DataTemplate>
    <StackLayout Margin="10" Padding="20" WidthRequest="150" BackgroundColor="#99FFFFFF">
      <Label FontSize="16" FontAttributes="Bold" Text="{Binding TimeAsString}" HorizontalOptions="
      <Image WidthRequest="100" HeightRequest="100" Aspect="AspectFit" HorizontalOptions="Center"
      <Label FontSize="14" FontAttributes="Bold" Text="{Binding Temperature, StringFormat='{0}° C'
      <Label FontSize="14" FontAttributes="Bold" Text="{Binding Description}" HorizontalOptions="C
    </StackLayout>
  </DataTemplate>
</BindableLayout.ItemTemplate>
```

TIP

The **`AspectFill`** *phrase, as a value of the* **`Aspect`** *property for* **`Image`**, *means that the whole image will always be visible and that the aspects will not be changed. The* **`AspectFit`** *phrase will also keep the aspect of an image, but the image can be zoomed into and out of and cropped so that it fills the whole* **`Image`** *element. The last value that* **`Aspect`** *can be set to,* **`Fill`**, *means that the image can be stretched or compressed to match the* **`Image`** *view to ensure that the aspect ratio is kept.*

Adding a toolbar item to refresh the weather data

To be able to refresh the data without restarting the app, we will add a **`Refresh`** button to the toolbar. **`MainViewModel`** is responsible for handling any logic that we want to perform, and we must expose any action as an **`ICommand`** bindable that we can bind to.

Let's start by creating the **`Refresh`** command method on **`MainViewModel`**:

1. Open the **`MainViewModel`** class.
2. Add a **`using`** declaration for **`CommunityToolkit.Mvvm.Input`**:

```
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using Weather.Models;
using Weather.Services;
```

3. Add a method called **`RefreshAsync`** that calls the **`LoadDataAsync`** method, as shown in the following code:

```
    public async Task RefreshAsync()
    {
        await LoadDataAsync();
    }
```

4. Since these methods are asynchronous, **`Refresh`** will return **`Task`**, and we can use **`async`** and **`await`** to call **`LoadDataAsync`** without blocking the UI thread.

5. Add a **RelayCommand** attribute to the **RefreshAsync** method to auto-generate the **ICommand** bindable property to the method:

```
[RelayCommand]
public async Task RefreshAsync()
{
    await LoadDataAsync();
}
```

Now that we have defined the **Refresh** command, we need to bind it to the user interface so that when the user clicks the toolbar button, the action will be executed.

To do this, perform the following steps:

1. Open the **MainView.xaml** file.
2. Download the **refresh.png** file from **https://raw.githubusercontent.com/PacktPublishing/MAUI-Projects-3rd-Edition/main/Chapter08/Weather/Resources/Images/refresh.png** and save it to the **Resources/Images** folder of the project.
3. Add a new **ToolbarItem** with the **Text** property set to **Refresh** to the **ToolbarItems** property of **ContentPage** and set the **IconImageSource** property to **refresh.png** (alternatively, you can set the **IconImageSource** property to the URL of the image and .NET MAUI will download the image).
4. Bind the **Command** property to the **Refresh** property in **MainViewModel**, as shown in the following code:

```
<ContentPage.ToolbarItems>
  <ToolbarItem IconImageSource=”refresh.png” Text=”Refresh” Command=”{Binding RefreshCommand}”
</ContentPage.ToolbarItems>
```

That’s all for refreshing the data. Now, we need some kind of indicator that the data is loading.

Adding a loading indicator

When we refresh the data, we want to show a loading indicator so that the user knows that something is happening. To do this, we will add **ActivityIndicator**, which is what this control is called in .NET MAUI. Let’s set this up:

1. Open the **MainViewModel** class.
2. Add a **Boolean** field called **isRefreshing** to **MainViewModel**.
3. Add the **ObservableProperty** attribute to **isRefreshingField** to generate the **IPropertyChanged** implementation.
4. Set the **IsRefreshing** property to **true** at the beginning of the **LoadDataAsync** method.
5. At the end of the **LoadDataAsync** method, set the **IsRefreshing** property to **false**, as shown in the following code:

```
    [ObservableProperty]
    private bool isRefreshing;
....// The rest of the code is omitted for brevity
public async Task LoadData()
{
    IsRefreshing = true;
....// The rest of the code is omitted for brevity
    IsRefreshing = false;
}
```

Now that we have added some code to **MainViewModel**, we need to bind the **IsRefreshing** property to a user interface element that will be displayed when the **IsRefreshing** property is **true**, as shown in the following steps:

1. In **MainView.xaml**, add **Frame** after **ScrollView** as the last element in **Grid**.
2. Bind the **IsVisible** property to the **IsRefreshing** method that we created in **MainViewModel**.
3. Set **HeightRequest** and **WidthRequest** to **100**.
4. Set **VerticalOptions** and **HorizontalOptions** to **Center** so that **Frame** will be in the middle of the view.
5. Set **BackgroundColor** to **#99000000** to set the background to white with a little bit of transparency.
6. Add **ActivityIndicator** to **Frame** with **Color** set to **Black** and **IsRunning** set to **True**, as shown in the following code:

```
<Frame IsVisible="{Binding IsRefreshing}" BackgroundColor="#99FFFFFF" WidthRequest="100" Height
  <ActivityIndicator Color="Black" IsRunning="True" />
</Frame>
```

This will create a spinner that will be visible while data is loading, which is a really good practice when creating any user interface. Now, we'll add a background image to make the app look a bit nicer.

Setting a background image

The last thing we will do to this view, for the moment, is add a background image. The image we will be using in this example is a result of a Google search for images that are free to use. Let's set this up:

1. Open the **MainView.xaml** file.
2. Set the **Background** property of **ScrollView** to **Transparent**.
3. Add an **Image** element in **Grid** with **UriImageSource** as the value of the **Source** property.
4. Set the **CachingEnabled** property to true and **CacheValidity** to **5**. This means that the image will be cached for **5** days.

NOTE

You could also set these properties if you used a URL for the **Refresh IconImageSource** *property to avoid downloading the image on every run of*

the app.

5. The XAML should now look as follows:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Weather.Views.Desktop.MainView"
    Title="{Binding City}">
  <ContentPage.ToolbarItems>
    <ToolbarItem Icon="refresh.png" Text="Refresh" Command="{Binding RefreshCommand}" />
  </ContentPage.ToolbarItems>
  <Grid>
    <Image Aspect="AspectFill">
      <Image.Source>
        <UriImageSource Uri="https://upload.wikimedia.org/wikipedia/commons/7/79/Solnedg%C3%A5n
      </Image.Source>
    </Image>
    <ScrollView BackgroundColor="Transparent">
<!-- The rest of the code is omitted for brevity -->
```

We can also set the URL directly in the **`Source`** property by using **`<Image Source="https://ourgreatimage.url" />`**. However, if we do this, we can't specify caching for the image.

With the desktop view complete, we need to consider how this page will look when we are running the app on a phone or tablet.

Creating the view for phones

Structuring content on a tablet and a desktop computer is very similar in many ways. On phones, however, we are much more limited in what we can do. Therefore, in this section, we will create a specific view for this app when it's used on phones. To do so, perform the following steps:

1. Create a new XAML-based **ContentPage** in the **`Views`** folder.
2. In the **`Views`** folder, create a new folder called **`Mobile`**.
3. Create a new **`.NET MAUI ContentPage (XAML)`** file called **`MainView`** in the **`Views\Mobile`** folder:

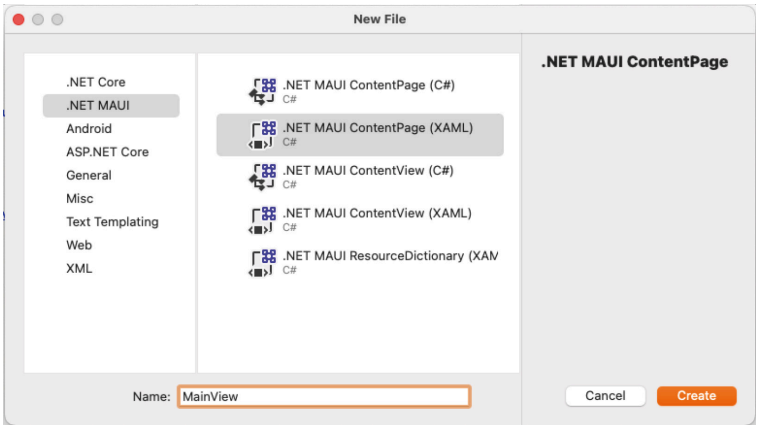

Figure 8.9 – Adding a .NET MAUI XAML ContentPage

4. Pass an instance of **`MainViewModel`** in the constructor of the view to set **`BindingContext`**, as shown in the following code:

```
public MainView (MainViewModel mainViewModel)
{
    InitializeComponent();
    BindingContext = mainViewModel;
}
```

Later, in the *Adding services and ViewModels to dependency injection* section, we will configure dependency injection to provide the instances for us.

To trigger the **LoadDataAsync** method in **MainViewModel**, call the **LoadDataAsync** method by overriding the **OnNavigatedTo** method on the main thread. We need to make sure that the call is executed on the UI thread since it will interact with the user interface.

To do this, perform the following steps:

1. Open the **MainView.xaml.cs** file in the **Views\Mobile** folder.
2. Create an override of the **OnNavigatedTo** method.
3. Add the highlighted code in the following snippet to the **OnNavigateTo** method:

```
    protected override void OnNavigatedTo(NavigatedToEventArgs args)
    {
        base.OnNavigatedTo(args);
        if (BindingContext is MainViewModel viewModel)
        {
            MainThread.BeginInvokeOnMainThread(async () =>
            {
                await viewModel.LoadDataAsync();
            });
        }
    }
```

In the **MainView** XAML file, add a binding for the **Title** property of **ContentPage** to the **City** property in **ViewModel**, as follows:

1. Open the **MainView.xaml** file in the **Views\Mobile** folder.
2. Add the **Title** binding to the **ContentPage** element, as highlighted in the following code snippet:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Weather.Views.Desktop.MainView"
    Title="{Binding City}">
```

In the next section, we will use **CollectionView** to display the weather data instead of using **FlexView**, as we did for the desktop view.

Using a grouped CollectionView

We could use **FlexLayout** for the phone's view, but because we want our user experience to be as good as possible, we will use **CollectionView** in-

stead. To get the headers for each day, we will use grouping for **`CollectionView`**. For **`FlexLayout`**, we had **`ScrollView`**, but for **`CollectionView`**, we don't need this because **`CollectionView`** can handle scrolling by default.

Let's continue creating the user interface for the phone's view:

1. Open the **`MainView.xaml`** file in the **`Views\Mobile`** folder.
2. Add **`CollectionView`** to the root of the page.
3. Set a binding to the **`Days`** property in **`MainViewModel`** for the **`ItemSource`** property.
4. Set **`IsGrouped`** to **`True`** to enable grouping in **`CollectionView`**.
5. Set **`BackgroundColor`** to **`Transparent`**, as shown in the following code:

```
<CollectionView ItemsSource="{Binding Days}" IsGrouped="True" BackgroundColor="Transparent">
</CollectionView>
```

To format how each header will look, we will create a **`DataTemplate`** property, as follows:

1. Add a **`DataTemplate`** property to the **`GroupHeaderTemplate`** property of **`CollectionView`**.
2. Add the content for the row to **`DataTemplate`**, as shown in the following code:

```
<CollectionView ItemsSource="{Binding Days}" IsGrouped="True" BackgroundColor="Transparent">
  <CollectionView.GroupHeaderTemplate>
    <DataTemplate>
      <ContentView Padding="15,5" BackgroundColor="#9F5010">
        <Label FontAttributes="Bold" TextColor="White" Text="{Binding DateAsString}" VerticalOp
      </ContentView>
    </DataTemplate>
  </CollectionView.GroupHeaderTemplate>
</CollectionView>
```

To format how each forecast will look, we will create a **`DataTemplate`** property, as we did with the group header. Let's set this up:

1. Add a **`DataTemplate`** property to the **`ItemTemplate`** property of **`CollectionView`**.
2. In **`DataTemplate`**, add a **`Grid`** property that contains four columns. Use the **`ColumnDefinition`** property to specify the width of the columns. The second column should be **`50`**; the other three will share the rest of the space. We will do this by setting **`Width`** to **`*`**.
3. Add the following content to **`Grid`**:

```
<CollectionView.ItemTemplate>
  <DataTemplate>
    <Grid Padding="15,10" ColumnSpacing="10" BackgroundColor="#99FFFFFF">
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="50" />
        <ColumnDefinition Width="*" />
```

```
        <ColumnDefinition Width="*" />
      </Grid.ColumnDefinitions>
      <Label FontAttributes="Bold" Text="{Binding TimeAsString}" VerticalOptions="Center" />
      <Image Grid.Column="1" HeightRequest="50" WidthRequest="50" Source="{Binding Icon}" Aspec
      <Label Grid.Column="2" Text="{Binding Temperature, StringFormat='{0}° C'}"
VerticalOptions="Center" />
      <Label Grid.Column="3" Text="{Binding Description}" VerticalOptions="Center" />
    </Grid>
  </DataTemplate>
</CollectionView.ItemTemplate>
```

Adding pull-to-refresh functionality

For the tablet and desktop versions of the view, we added a button to the toolbar to refresh the weather forecast. In the phone version of the view, however, we will add pull-to-refresh functionality, which is a common way to refresh content in a list of data. **`CollectionView`** in .NET MAUI has no built-in support for pull-to-refresh as **`ListView`** has.

Instead, we can use **`RefreshView`**. **`RefreshView`** can be used to add pull-to-refresh behavior to any control. Let's set this up:

1. Go to **`Views\Mobile\MainView.xaml`**.
2. Wrap **`CollectionView`** inside **`RefreshView`**.
3. Bind the **`RefreshCommand`** property in **`MainViewModel`** to the **`Command`** property of **`RefreshView`** to trigger a refresh when the user performs a pull-to-refresh gesture.
4. To show a loading icon when the refresh is in progress, bind the **`IsRefreshing`** property in **`MainViewModel`** to the **`IsRefreshing`** property of **`RefreshView`**. When we are setting this up, we will also get a loading indicator when the initial load is running, as shown in the following code:

```
<RefreshView Command="{Binding Refresh}" IsRefreshing="{Binding IsRefreshing}">
  <CollectionView ItemsSource="{Binding Days}" IsGrouped="True" BackgroundColor="Transparent">
....
  </CollectionView>
</RefreshView>
```

That concludes the views for the moment. Now, let's wire them up into dependency injection so that we can see our work.

Adding services and ViewModels to dependency injection

For our views to get an instance of **`MainViewModel`** and **`MainViewModel`** to get an instance of **`OpenWeatherMapWeatherService`**, we need to add them to dependency injection. Let's set this up:

1. Open **`MauiProgram.cs`**.
2. Add the following highlighted code:

```
#if DEBUG
```

```
    builder.Logging.AddDebug();
#endif
    builder.Services.AddSingleton<IWeatherService, OpenWeatherMapWeatherService>();
    builder.Services.AddTransient<MainViewModel, MainViewModel>();
    return builder.Build();
```

In the next section, we will add the navigation to the views based on the device's form factor.

Navigating to different views based on the form factor

We now have two different views that should be loaded in the same place in the app. **`Weather.Views.Desktop.MainView`** should be loaded if the app is running on a tablet or a desktop, while **`Weather.Views.Mobile.MainView`** should be loaded if the app is running on a phone.

The **`Device`** class in .NET MAUI has a static **`Idiom`** property that we can use to check which form factor the app is running on. The value of **`Idiom`** can be **`Phone`**, **`Tablet`**, **`Desktop`**, **`Watch`**, or **`TV`**. Because we only have one view in this app, we could have used an **`if`** statement when we were setting **`MainPage`** in **`App.xaml.cs`** and checked what the **`Idiom`** value was.

Since we are only ever going to need one view, we can register just the view that we need in dependency injection – all we need is a common type to register the views with. Let's create a new interface that our views will implement:

1. Create a new interface in the **`Views`** folder named **`IMainView`**.
   We won't add any additional properties or methods to the interface – we'll just use it as a marker.
2. Open **`Views\Desktop\MainView.xaml.cs`** and add the **`IMainView`** interface to the class:
   ```
   public partial class MainView : ContentPage, IMainView
   ```
3. Open **`Views\Mobile\MainView.xaml.cs`** and add the **`IMainView`** interface to the class:
   ```
   public partial class MainView : ContentPage, IMainView
   ```

Now that we have a common interface, we can register the views with dependency injection:

1. Open the **`MauiProgram.cs`** file.
2. Add the following code:
   ```
   #if DEBUG
       builder.Logging.AddDebug();
   #endif
       builder.Services.AddSingleton<IWeatherService, OpenWeatherMapWeatherService>();
       builder.Services.AddTransient<MainViewModel, MainViewModel>();
   ```

```
if (DeviceInfo.Idiom == DeviceIdiom.Phone)
{
    builder.Services.AddTransient<IMainView, Views.Mobile.MainView>();
}
else
{
    builder.Services.AddTransient<IMainView, Views.Desktop.MainView>();
}
return builder.Build();
```

With these changes, we can now test our application. If you run your app, you should see something like the following:

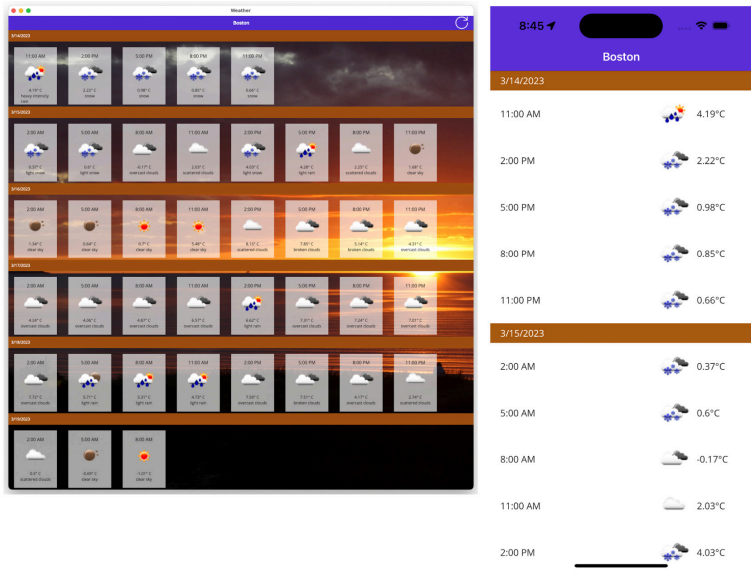

Figure 8.10 – App running on macOS and iOS

Next, let's update the desktop views to handle resizing properly by using `VisualStateManager`.

Handling states with VisualStateManager

`VisualStateManager` is a way to make changes in the UI from the code. We can define states and set values for selected properties to apply to a specific state. `VisualStateManager` can be useful in cases where we want to use the same view for devices with different screen resolutions. It was first introduced in **UWP** to make it easier to create Windows 10 applications for multiple platforms. This was because Windows 10 could run on Windows Phone, as well as on desktops and tablets (the operating system was called Windows 10 Mobile). However, Windows Phone has now been deprecated. `VisualStateManager` is interesting for us as .NET MAUI developers, especially when both iOS and Android can run on both phones and tablets.

In this project, we will use it to make a `forecast` item bigger when the app is running in landscape mode on a tablet or a desktop. We will also make the weather icon bigger. Let's set this up:

1. Open the `Views\Desktop\MainView.xaml` file.

2. In the first **FlexLayout** and **DataTemplate**, insert a **VisualStateManager.VisualStateGroups** element into the first **StackLayout**:

```
<StackLayout Margin="10" Padding="20" WidthRequest="150" BackgroundColor="#99FFFFFF">
  <VisualStateManager.VisualStateGroups>
    <VisualStateGroup>
    </VisualStateGroup>
  </VisualStateManager.VisualStateGroups>
......
</StackLayout>
```

Regarding **VisualStateGroup**, we should add two states, as follows:

1. Add a new **VisualState** called **Portrait** to **VisualStateGroup**.
2. Create a setter in **VisualState** and set **WidthRequest** to **150**.
3. Add another **VisualState** called **Landscape** to **VisualStateGroup**.
4. Create a setter in **VisualState** and set **WidthRequest** to **200**, as shown in the following code:

```
<VisualStateGroup>
  <VisualState Name="Portrait">
    <VisualState.Setters>
      <Setter Property="WidthRequest" Value="150" />
      </VisualState.Setters>
    </VisualState>
  <VisualState Name="Landscape">
    <VisualState.Setters>
      <Setter Property="WidthRequest" Value="200" />
    </VisualState.Setters>
  </VisualState>
</VisualStateGroup>
```

We also want the icons in a forecast item to be bigger when the item itself is bigger. To do this, we will use **VisualStateManager** again. Let's set this up:

1. Insert a **VisualStateManager.VisualStateGroups** element into the second **FlexLayout** and in the **Image** element in **DataTemplate**.
2. Add **VisualState** for both **Portrait** and **Landscape**.
3. Add setters to the states to set **WidthRequest** and **HeightRequest**. The value should be **100** in the **Portrait** state and **150** in the **Landscape** state, as shown in the following code:

```
<Image WidthRequest="100" HeightRequest="100" Aspect="AspectFit" HorizontalOptions="Center" Sou
  <VisualStateManager.VisualStateGroups>
    <VisualStateGroup>
      <VisualState Name="Portrait">
        <VisualState.Setters>
          <Setter Property="WidthRequest" Value="100" />
          <Setter Property="HeightRequest" Value="100" />
        </VisualState.Setters>
      </VisualState>
      <VisualState Name="Landscape">
        <VisualState.Setters>
```

```
            <Setter Property="WidthRequest" Value="150" />
            <Setter Property="HeightRequest" Value="150" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
</Image>
```

Creating a behavior to set state changes

With **Behavior**, we can add functionality to controls without having to subclass them. With behaviors, we can also create more reusable code than we could if we subclassed a control. The more specific **Behavior** we create, the more reusable it will be. For example, **Behavior** that inherits from **Behavior<View>** could be used on all controls, but **Behavior** that inherits from **Button** can only be used for buttons. Because of this, we always want to create behaviors with a less specific base class.

When we create **Behavior**, we need to override two methods: **OnAttached** and **OnDetachingFrom**. It is really important to remove event listeners in the **OnDeattached** method if we have added them to the **OnAttached** method. This will make the app use less memory. It is also important to set values back to the values that they had before the **OnAppearing** method ran; otherwise, we might see some strange behavior, especially if the behavior is in a **CollectionView** or **ListView** view that is reusing cells.

In this app, we will create a behavior for **FlexLayout**. This is because we can't set the state of an item in **FlexLayout** from the code-behind. We could have added some code to check whether the app runs in portrait or landscape in **FlexLayout**, but if we use **Behavior** instead, we can separate that code from **FlexLayout** so that it will be more reusable. Perform the following steps to do so:

1. Create a new folder called **Behaviors**.
2. Create a new class called **FlexLayoutBehavior**.
3. Add **Behavior<FlexLayoutView>** as a base class.
4. Create a **private** field of the **FlexLayout** type called **view**.
5. The code should look as follows:

```
using System;
namespace Weather.Behaviors;
public class FlexLayoutBehavior : Behavior<FlexLayout>
{
    private FlexLayout view;
}
```

FlexLayout is a class that inherits from the **Behavior<FlexLayout>** base class. This will give us the ability to override some virtual methods that will be called when we attach and detach the behavior from a **FlexLayout** class.

But first, we need to create a method that will handle the change in state. Perform the following steps to do so:

1. Open the **FlexlayoutBehavior.cs** file.
2. Create a **private** method called **SetState**. This method will have a **VisualElement** value and a **string** argument.
3. Call **VisualStateManager.GoToState** and pass the parameters to it.
4. If the view is of the **Layout** type, there might be child elements that also need to get the new state. To do that, we will loop through all the children of the layout. Instead of just setting the state directly to the children, we will call the **SetState** method, which is the method that we are already inside. The reason for this is that some of the children may have their own children:

```
private void SetState(VisualElement view, string state)
{
    VisualStateManager.GoToState(view, state);
    if (view is Layout layout)
    {
        foreach (VisualElement child in layout.Children)
        {
            SetState(child, state);
        }
    }
}
```

Now that we have created the **SetState** method, we need to write a method that uses it and determines what state to set. Perform the following steps to do so:

1. Create a **private** method called **UpdateState**.
2. Run the code on **MainThread** to check whether the app is running in portrait or landscape mode.
3. Create a variable called **page** and set its value to **Application.Current.MainPage**.
4. Check whether **Width** is larger than **Height**. If this is **true**, set the **VisualState** property on the **view** variable to **Landscape**. If this is **false**, set the **VisualState** property on the **view** variable to **Portrait**, as shown in the following code:

```
private void UpdateState()
{
    MainThread.BeginInvokeOnMainThread(() =>
    {
        var page = Application.Current.MainPage;
        if (page.Width > page.Height)
        {
            SetState(view, “Landscape”);
            return;
        }
        SetState(view, “Portrait”);
    });
}
```

With that, the **UpdateState** method has been added. Now, we need to override the **OnAttachedTo** method, which will be called when the behavior is added to **FlexLayout**. When it is, we want to update the state by calling this method and hook it up to the **SizeChanged** event of **MainPage** so that when the size changes, we will update the state again.

Let's set this up:

1. Open the **FlexLayoutBehavior** file.
2. Override the **OnAttachedTo** method from the base class.
3. Set the **view** property to the parameter from the **OnAttachedTo** method.
4. Add an event listener to **Application.Current.MainPage.SizeChanged**. In the event listener, add a call to the **UpdateState** method, as shown in the following code:

```
protected override void OnAttachedTo(FlexLayout view)
{
    this.view = view;
    base.OnAttachedTo(view);
    UpdateState();
    Application.Current.MainPage.SizeChanged += MainPage_SizeChanged;
}
void MainPage_SizeChanged(object sender, EventArgs e)
{
    UpdateState();
}
```

When we remove behaviors from a control, it's very important to also remove any event handlers from it to avoid memory leaks, and in the worst case, the app crashing. Let's do this:

1. Open the **FlexLayoutBehavior.cs** file.
2. Override **OnDetachingFrom** from the base class.
3. Remove the event listener from **Application.Current.MainPage.SizeChanged**.
4. Set the **view** field to **null**, as shown in the following code:

```
protected override void OnDetachingFrom(FlexLayout view)
{
    base.OnDetachingFrom(view);
    Application.Current.MainPage.SizeChanged -= MainPage_SizeChanged;
    this.view = null;
}
```

Perform the following steps to add **behavior** to the view:

1. Open the **MainView.xaml** file inside the **Views/Desktop** folder.
2. Import the **Weather.Behaviors** namespace, as shown in the following code:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
```

```
    xmlns:behaviors="clr-namespace:Weather.Behaviors"
    x:Class="Weather.Views.Desktop.MainView"
    Title="{Binding City}">
```

The last thing we will do is add **`FlexLayoutBehavior`** to the second **`FlexLayout`**, as shown in the following code:

```
<FlexLayout ItemsSource="{Binding Items}" Wrap="Wrap" JustifyContent="Start" AlignItems="Start">
  <FlexLayout.Behaviors>
    <behaviors:FlexLayoutBehavior />
  </FlexLayout.Behaviors>
<FlexLayout.ItemsTemplate>
```

Congratulations – that is a wrap on the weather app!

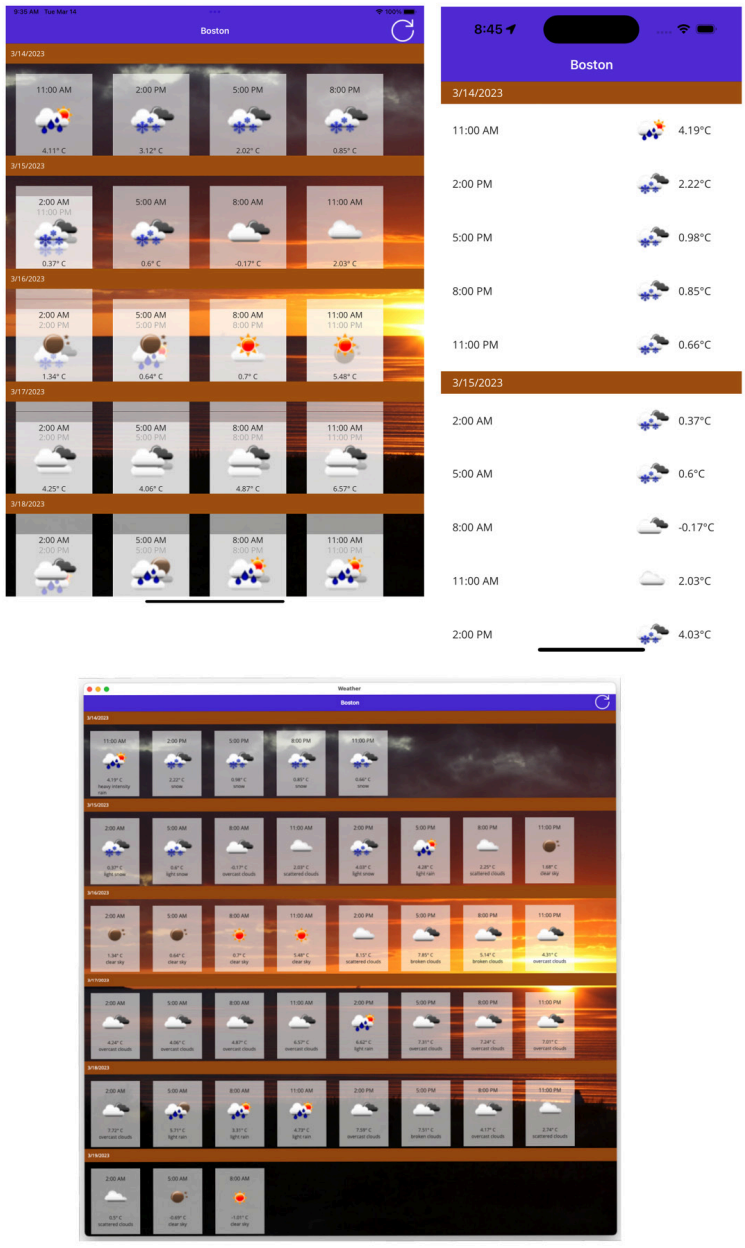

Figure 8.11 – Completed app on tablet, phone, and desktop

Summary

In this chapter, we successfully created an app for four different operating systems – iOS, macOS, Android, and Windows – and three different form factors – phones, tablets, and desktop computers. To create a good user experience on all platforms and form factors, we used `FlexLayout` and `VisualStateManager`. We also learned how to handle different views for different form factors, as well as how to use `Behaviors`.

The next app we will build will be a game with real-time communication. In the next chapter, we will take a look at how we can use the **SignalR** service in **Azure** as the backend game service.